



Java 核心技术(进阶)

第四章 高级文件处理

第六节 docx简介及解析

华东师范大学 陈良育



Docx 简介

- 以Microsoft Office的doc/docx为主要处理对象。
- Word2003（包括）之前都是doc，文档格式不公开。
- Word2007（包括）之后都是docx，遵循XML路线，文档格式公开。
- docx为主要研究对象
 - 文字样式
 - 表格
 - 图片
 - 公式



Docx 功能和处理

- 常见功能

- docx解析
- docx生成（完全生成，模板加部分生成：套打）

- 处理的第三方库

- Jacob, COM4J (Windows 平台)
- **POI**, docx4j, OpenOffice/Libre Office SDK (**免费**)
- Aspose (收费)
- 一些开源的OpenXML的包。

POI



- Apache POI

- Apache 出品，必属精品， poi.apache.org
- 可处理docx, xlsx, pptx, visio等office套件
- 纯Java工具包，无需第三方依赖
- 主要类

- XWPFDocument 整个文档对象
- XWPFParagraph 段落
- XWPFRun 一个片段(字体样式相同的一段)
- XWPFPicture 图片
- XWPFTable 表格

心中有一尊神。

①那天，街上很冷，太阳淡淡地照着，薄得如纸，母亲引着 8 岁的我走过清冷的街道。我吸溜着鼻涕，拉着母亲的手，另一只手上拿着一个烤白薯。烤白薯冒着缕缕热气，香味很有诱惑力地飘散到空气中。阳光，也仿佛染上了诱人的香味。

②我捏着烤白薯，看着里面黄亮亮的瓤儿，咬了一口，一种软乎乎的幸福直钻入我的五脏六腑中。那种香气，至今想起来，仿佛还荡漾在我的记忆里，缭绕不散。



Doc/Docx处理总结

- 不同的Office工具，产生出来的docx文件格式不兼容
- 不同的第三方包，能够解析和生成的内容也不同，使用的类也不同
- doc/docx功能非常非常多，第三方包不是万能的，也存在无法解析的情况
- API很多，需要多查询、多练习

代码(1) TextRead.java



```
package docx;

import java.io.FileInputStream;

public class TextRead {

    public static void main(String[] args) throws Exception {
        readDocx();
    }

    public static void readDocx() throws Exception {
        InputStream is;
        is = new FileInputStream("test.docx");
        XWPFDocument xwpf = new XWPFDocument(is);

        List<IBodyElement> ibs= xwpf.getBodyElements();
    }
}
```

代码(2) TextRead.java



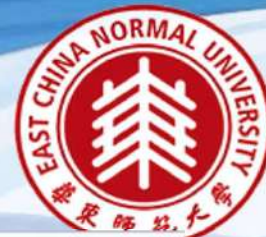
```
for(IBodyElement ib:ibs)
{
    BodyElementType bet = ib.getElementType();
    if(bet== BodyElementType.TABLE)
    {
        //表格
        System.out.println("table" + ib.getPart());
    }
    else
    {
        //段落
        XWPFPParagraph para = (XWPFPParagraph) ib;
        System.out.println("It is a new paragraph....The indentation is "
            + para.getFirstLineIndent() + "," + para.getIndentationFirstLine() );
        //System.out.println(para.getCTP().xmlText());

        List<XWPFRun> res = para.getRuns();
        //System.out.println("run");
        if(res.size()<=0)
        {
            System.out.println("empty line");
        }
    }
}
```


代码(3) TextRead.java



```
for(XWPFRun re: res)
{
    if(null == re.text() || re.text().length() <= 0)
    {
        if(re.getEmbeddedPictures().size() > 0)
        {
            System.out.println("image***" + re.getEmbeddedPictures().size());
        } else
        {
            System.out.println("objects:" + re.getCTR().getObjectList().size());
            if(re.getCTR().xmlText().indexOf("instrText") > 0) {
                System.out.println("there is an equation field");
            }
            else
            {
                //System.out.println(re.getCTR().xmlText());
            }
        }
    }
    else
    {
        System.out.println("==" + re.getCharacterSpacing() + re.text());
    }
}
is.close();
}
```

代码(4) ImageRead.java

```
package docx;
```

* 本类 完成docx的图片读取工作

```
import org.apache.poi.openxml4j.exceptions.InvalidFormatException;
```

```
public class ImageRead {
```

```
    public static void imageRead() throws IOException, InvalidFormatException {  
        File docFile = new File("simple.docx");
```

```
        XWPFDocument doc = new XWPFDocument(OPCPackage.openOrCreate(docFile));
```



代码(5) ImageRead.java

```
int i = 0;
for (XWPFParagraph p : doc.getParagraphs()) {
    for (XWPFRun run : p.getRuns()) {
        System.out.println("a new run");
        for (XWPFPicture pic : run.getEmbeddedPictures()) {
            System.out.println(pic.getCTPicture().xmlText());
            //image EMU(English Metric Unit)
            System.out.println(pic.getCTPicture().getSpPr().getXfrm().getExt().getCx());
            System.out.println(pic.getCTPicture().getSpPr().getXfrm().getExt().getCy());
            //image 显示大小 以厘米为单位
            System.out.println(pic.getCTPicture().getSpPr().getXfrm().getExt().getCx()/360000.0);
            System.out.println(pic.getCTPicture().getSpPr().getXfrm().getExt().getCy()/360000.0);
            int type = pic.getPictureData().getPictureType();
            byte [] img = pic.getPictureData().getData();
            BufferedImage bufferedImage= ImageRead.byteArrayToImage(img);
            System.out.println(bufferedImage.getWidth());
            System.out.println(bufferedImage.getHeight());
            String extension = "";
```


代码(6) ImageRead.java



```
switch(type)
{
    case Document.PICTURE_TYPE_EMF: extension = ".emf";
                                     break;
    case Document.PICTURE_TYPE_WMF: extension = ".wmf";
                                     break;
    case Document.PICTURE_TYPE_PICT: extension = ".pic";
                                     break;
    case Document.PICTURE_TYPE_PNG: extension = ".png";
                                     break;
    case Document.PICTURE_TYPE_DIB: extension = ".dib";
                                     break;
    default: extension = ".jpg";
}
//outputFile = new File ( );
//BufferedImage image = ImageIO.read(new File(img));
//ImageIO.write(image , "png", outputfile);
FileOutputStream fos = new FileOutputStream("test" + i + extension);
fos.write(img);
fos.close();
i++;
}
}
}
```

代码(7) ImageRead.java



```
public static BufferedImage byteArrayToImage(byte[] bytes){
    BufferedImage bufferedImage=null;
    try {
        InputStream inputStream = new ByteArrayInputStream(bytes);
        bufferedImage = ImageIO.read(inputStream);
    } catch (IOException ex) {
        System.out.println(ex.getMessage());
    }
    return bufferedImage;
}

public static void main(String[] args) throws Exception {
    imageRead();
}
}
```




代码(8) ImageWrite.java

```
package docx;
```

```
* 本类完成docx的图片保存工作
```

```
import org.apache.poi.util.Units;
```

```
public class ImageWrite {
```

```
    public static void main(String[] args) throws Exception {
```

```
        XWPFDocument doc = new XWPFDocument();
```

```
        XWPFParagraph p = doc.createParagraph();
```

```
        XWPFRun r = p.createRun();
```

```
        String[] imgFiles = new String[2];
```

```
        imgFiles[0] = "c:/temp/ecnu.jpg";
```

```
        imgFiles[1] = "c:/temp/shida.jpg";
```



代码(9) ImageWrite.java

```
for(String imgFile : imgFiles) {
    int format;

    if(imgFile.endsWith(".emf")) format = XWPFDocument.PICTURE_TYPE_EMF;
    else if(imgFile.endsWith(".wmf")) format = XWPFDocument.PICTURE_TYPE_WMF;
    else if(imgFile.endsWith(".pict")) format = XWPFDocument.PICTURE_TYPE_PICT;
    else if(imgFile.endsWith(".jpeg") || imgFile.endsWith(".jpg")) format = XWPFDocument.PICTURE_TYPE_JPEG;
    else if(imgFile.endsWith(".png")) format = XWPFDocument.PICTURE_TYPE_PNG;
    else if(imgFile.endsWith(".dib")) format = XWPFDocument.PICTURE_TYPE_DIB;
    else if(imgFile.endsWith(".gif")) format = XWPFDocument.PICTURE_TYPE_GIF;
    else if(imgFile.endsWith(".tiff")) format = XWPFDocument.PICTURE_TYPE_TIFF;
    else if(imgFile.endsWith(".eps")) format = XWPFDocument.PICTURE_TYPE_EPS;
    else if(imgFile.endsWith(".bmp")) format = XWPFDocument.PICTURE_TYPE_BMP;
    else if(imgFile.endsWith(".wpg")) format = XWPFDocument.PICTURE_TYPE_WPG;
    else {
        System.err.println("Unsupported picture: " + imgFile +
            ". Expected emf|wmf|pict|jpeg|png|dib|gif|tiff|eps|bmp|wpg");
        continue;
    }
}
```

代码(10) ImageWrite.java



```
r.setText(imgFile);  
r.addBreak();  
r.addPicture(new FileInputStream(imgFile), format, imgFile, Units.toEMU(200), Units.toEMU(200)); // 200x200 pixels  
r.addBreak(BreakType.PAGE);  
}
```

```
FileOutputStream out = new FileOutputStream("images.docx");  
doc.write(out);  
out.close();  
}
```

```
}
```




代码(11) TableRead.java

```
package docx;

/* 本类完成docx的表格内容读取 */
import java.io.FileInputStream;

public class TableRead {
    public static void main(String[] args) throws Exception {
        testTable();
    }

    public static void testTable() throws Exception {
        InputStream is = new FileInputStream("simple2.docx");
        XWPFDocument xwpf = new XWPFDocument(is);
        List<XWPFParagraph> paras = xwpf.getParagraphs();
        //List<POIXMLDocumentPart> pdps = xwpf.getRelations();
    }
}
```

代码(12) TableRead.java



```
List<IBodyElement> ibs= xwpf.getBodyElements();
for(IBodyElement ib:ibs)
{
    BodyElementType bet = ib.getElementType();
    if(bet== BodyElementType.TABLE)
    {
        //表格
        System.out.println("table" + ib.getPart());
        XWPFTable table = (XWPFTable) ib;
        List<XWPFTableRow> rows=table.getRows();
        //读取每一行数据
        for (int i = 0; i < rows.size(); i++) {
            XWPFTableRow row = rows.get(i);
            //读取每一列数据
            List<XWPFTableCell> cells = row.getTableCells();
            for (int j = 0; j < cells.size(); j++) {
                XWPFTableCell cell=cells.get(j);
                System.out.println(cell.getText());
                List<XWPFPParagraph> cps = cell.getParagraphs();
                System.out.println(cps.size());
            }
        }
    }
}
```


代码(13) TableRead.java



```
else
{
    //段落
    XWPFParagraph para = (XWPFParagraph) ib;
    System.out.println("It is a new paragraph....The indention is "
        + para.getFirstLineIndent() + "," + para.getIndentationFirstLine() + ","
        + para.getIndentationHanging() + "," + para.getIndentationLeft() + ","
        + para.getIndentationRight() + "," + para.getIndentFromLeft() + ","
        + para.getIndentFromRight() + "," + para.getAlignment().getValue());

    //System.out.println(para.getAlignment());
    //System.out.println(para.getRuns().size());

    List<XWPFRun> res = para.getRuns();
    System.out.println("run");
    if(res.size()<=0)
    {
        System.out.println("empty line");
    }
}
```


代码(14) TableRead.java



```
for(XWPFRun re: res)
{
    if(null == re.text() || re.text().length() <= 0)
    {
        if(re.getEmbeddedPictures().size() > 0)
        {
            System.out.println("image***" + re.getEmbeddedPictures().size());
        }
        else
        {
            System.out.println("objects:" + re.getCTR().getObjectList().size());
            System.out.println(re.getCTR().xmlText());
        }
    }
    else
    {
        System.out.println("===" + re.text());
    }
}
}
is.close();
}
```

代码(15) TableWrite.java



```
package docx;

* 本类测试写入表格

import java.io.FileOutputStream;

public class TableWrite {

    public static void main(String[] args) throws Exception {
        try {
            createSimpleTable();
        }
        catch(Exception e) {
            System.out.println("Error trying to create simple table.");
            throw(e);
        }
    }
}
```

代码(16) TableWrite.java



```
public static void createSimpleTable() throws Exception {  
    XWPFDocument doc = new XWPFDocument();  
  
    try {  
        XWPFTable table = doc.createTable(3, 3);  
  
        table.getRow(1).getCell(1).setText("表格示例");  
  
        XWPFParagraph p1 = table.getRow(0).getCell(0).getParagraphs().get(0);  
  
        XWPFRun r1 = p1.createRun();  
        r1.setBold(true);  
        r1.setText("The quick brown fox");  
        r1.setItalic(true);  
        r1.setFontFamily("Courier");  
        r1.setUnderline(UnderlinePatterns.DOT_DOT_DASH);  
        r1.setTextPosition(100);  
    }  
}
```


代码(17) TableWrite.java



```
table.getRow(2).getCell(2).setText("only text");

OutputStream out = new FileOutputStream("simpleTable.docx");
try {
    doc.write(out);
} finally {
    out.close();
}
} finally {
    doc.close();
}
}
```

代码(18) TemplateTest.java



```
public class TemplateTest {  
  
    public static void main(String[] args) throws InvalidFormatException, FileNotFoundException, IOException {  
        XWPFDocument doc = openDocx("template.docx");//导入模板文件  
        Map<String, Object> params = new HashMap<>();//文字类 key-value  
        params.put("${name}", "Tom");  
        params.put("${sex}", "男");  
        Map<String,String> picParams = new HashMap<>();//图片类 key-url  
        picParams.put("${pic}", "c:/temp/ecnu.jpg");  
        List<IBodyElement> ibes = doc.getBodyElements();  
        for (IBodyElement ib : ibes) {  
            if (ib.getElementType() == BodyElementType.TABLE) {  
                replaceTable(ib, params, picParams, doc);  
            }  
        }  
        writeDocx(doc, new FileOutputStream("template2.docx"));//输出  
    }  
}
```

代码(19) TemplateTest.java



```
public static XWPFDocument openDocx(String url) {
    InputStream in = null;
    try {
        in = new FileInputStream(url);
        XWPFDocument doc = new XWPFDocument(in);
        return doc;
    } catch (FileNotFoundException e) {
        e.printStackTrace();
    } catch (IOException e) {
        e.printStackTrace();
    } finally {
        if (in != null) {
            try {
                in.close();
            } catch (IOException e) {
                // TODO Auto-generated catch block
                e.printStackTrace();
            }
        }
    }
    return null;
}
```


代码(20) TemplateTest.java



```
public static void writeDocx(XWPFDocument outDoc, OutputStream out) {
    try {
        outDoc.write(out);
        out.flush();
        if (out != null) {
            try {
                out.close();
            } catch (IOException e) {
                e.printStackTrace();
            }
        }
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private static Matcher matcher(String str) {
    Pattern pattern = Pattern.compile("\\$\\{(.+?)\\}", Pattern.CASE_INSENSITIVE);
    Matcher matcher = pattern.matcher(str);
    return matcher;
}
```

代码(21) TemplateTest.java



```
public static void replacePic(XWPFRun run, String imgFile, XWPFDocument doc) throws Exception {
    int format;
    if (imgFile.endsWith(".emf"))
        format = Document.PICTURE_TYPE_EMF;
    else if (imgFile.endsWith(".wmf"))
        format = Document.PICTURE_TYPE_WMF;
    else if (imgFile.endsWith(".pict"))
        format = Document.PICTURE_TYPE_PICT;
    else if (imgFile.endsWith(".jpeg") || imgFile.endsWith(".jpg"))
        format = Document.PICTURE_TYPE_JPEG;
    else if (imgFile.endsWith(".png"))
        format = Document.PICTURE_TYPE_PNG;
    else if (imgFile.endsWith(".dib"))
        format = Document.PICTURE_TYPE_DIB;
    else if (imgFile.endsWith(".gif"))
        format = Document.PICTURE_TYPE_GIF;
    else if (imgFile.endsWith(".tiff"))
        format = Document.PICTURE_TYPE_TIFF;
    else if (imgFile.endsWith(".eps"))
        format = Document.PICTURE_TYPE_EPS;
    else if (imgFile.endsWith(".bmp"))
        format = Document.PICTURE_TYPE_BMP;
    else if (imgFile.endsWith(".wpg"))
        format = Document.PICTURE_TYPE_WPG;
    ,
    ,
```



代码(22) TemplateTest.java

```
else {  
    System.err.println(  
        "Unsupported picture: " + imgFile + ". Expected emf|wmf|pict|jpeg|png|dib|gif|tiff|eps|bmp|wpg");  
    return;  
}  
if(imgFile.startsWith("http")||imgFile.startsWith("https")){  
    run.addPicture(new URL(imgFile).openConnection().getInputStream(), format, "rpic",Units.toEMU(100),Units.toEMU(100));  
}else{  
    run.addPicture(new FileInputStream(imgFile), format, "rpic",Units.toEMU(100),Units.toEMU(100));  
}  
}
```


代码(23) TemplateTest.java



```
public static void replaceTable(IBodyElement para ,Map<String, Object> params,
    Map<String, String> picParams, XWPFDocument indoc)
    throws Exception {
    Matcher matcher;
    XWPFTable table;
    List<XWPFTableRow> rows;
    List<XWPFTableCell> cells;
    table = (XWPFTable) para;
    rows = table.getRows();
    for (XWPFTableRow row : rows) {
        cells = row.getTableCells();
        int cellsize = cells.size();
        int cellcount = 0;
        for(cellcount = 0; cellcount<cellsize;cellcount++){
            XWPFTableCell cell = cells.get(cellcount);
            String runtext = "";
            List<XWPFParagraph> ps = cell.getParagraphs();
            for (XWPFParagraph p : ps) {
                for(XWPFRun run : p.getRuns()){
                    runtext = run.text();
                    matcher = matcher(runtext);
```

代码(24) TemplateTest.java



```
if (matcher.find()) {
    if (picParams != null) {
        for (String pickey : picParams.keySet()) {
            if (matcher.group().equals(pickey)) {
                run.setText("",0);
                replacePic(run, picParams.get(pickey), indoc);
            }
        }
    }
    if (params != null) {
        for (String pickey : params.keySet()) {
            if (matcher.group().equals(pickey)) {
                run.setText(params.get(pickey)+"",0);
            }
        }
    }
}
}
```



谢谢!